AI Agent Orchestrated Phishing Detection Using Natural Language Processing and Machine Learning

Tolga Üner
Computer Engineering Dept.
Dokuz Eylul University
İzmir, Türkiye
tolga.uner@ogr.deu.edu.tr

Gökhan Dalkılıç
Computer Engineering Dept.
Dokuz Eylul University
İzmir, Türkiye
gokhan.dalkilic@deu.edu.tr

***Abstract*—Phishing emails continue to be a persistent threat in the cybersecurity landscape. Addressing the limitations of manual detection workflows, this paper introduces an agent-orchestrated framework that autonomously constructs, executes, and evaluates phishing detection pipelines using Machine Learning (ML), Natural Language Processing (NLP), and Deep Learning (DL). The Model Context Protocol (MCP) architecture serves as the foundation for the system, where an Artificial Intelligence (AI) agent generates and evaluates ML/DL pipelines under predefined constraints. A self-healing mechanism is introduced to detect and repair runtime errors, improving system robustness. The framework is evaluated on two publicly available datasets using both ML models and DL architectures. The proposed system achieves up to 98.76% accuracy, according to the results with hybrid deep learning models providing the best performance. These findings demonstrate that agent-guided automation enables reliable and reproducible phishing detection while reducing manual effort.**

***Index Terms*—Phishing Detection, Text Analysis, Machine Learning Methods, Deep Neural Networks, Model Context Protocol, Cybersecurity, AI Agent**

I. Introduction

One of the most dangerous cyberattacks is phishing, which targets individuals or businesses with fake emails intended to obtain sensitive data, including login credentials, financial details, and personal information.

To combat this threat, automated email classification systems have become essential. In recent years, Natural Language Processing (NLP) methods have been widely used to analyze email content and extract meaningful patterns. By transforming raw text into structured representations, NLP enables Machine Learning (ML) and Deep Learning (DL) models to classify emails as phishing or legitimate. ML approaches, such as Logistic Regression and Support Vector Machines, typically rely on feature extraction methods like TF–IDF, while DL approaches, including Convolutional Neural Networks (CNNs), and Recurrent Neural Networks (RNNs), learn hierarchical representations directly from text data.

Although these approaches have achieved high accuracy, they often require substantial manual effort in designing preprocessing pipelines, selecting appropriate models, tuning hyperparameters, and adapting workflows to different datasets. This process can be time-consuming, error-prone, and difficult to reproduce consistently across experiments.

This research proposes an agent-guided framework for automated phishing email detection to overcome these challenges. The system is built on a Model Context Protocol (MCP)-based architecture, which enables an AI agent to dynamically generate, execute, and evaluate ML and DL pipelines under predefined constraints. In addition to automation, the framework introduces a self-healing mechanism that captures runtime errors and iteratively repairs generated code, improving robustness and reducing the need for manual intervention.

The proposed approach aims to provide a reproducible and scalable solution for phishing detection by integrating NLP-based text processing with automated ML and DL workflows, while ensuring consistent evaluation across multiple datasets.

II. Related Works

Paradkar explored which machine learning and deep learning methods are best for detecting Phishing Emails. This study uses the ENRON Corpus, which has over 500,000 emails. The research utilized a sample of 20,000 emails, of which 8,336 were phishing, and 11,664 were verified as safe. After that, the data was preprocessed, and then the study evaluated models categorized into machine learning classifiers, such as Logistic Regression, Naive Bayes, SVM, and Decision Trees, and deep learning architectures, including CNN, LSTM, and Bidirectional-LSTM. The machine learning techniques gave worse results than the deep learning techniques [1].

Alhogail and Alsabih investigated detecting phishing emails using machine learning and Natural Language Processing. To detect phishing emails the study goes through three phases. The first phase is collecting data and preparing it for training and testing. This study used a fraud dataset that is available to the public. Normal emails are labeled as 0, and phishing emails are labeled as 1. The dataset contains 3685 phishing emails and 4894 normal emails. After preparing the data they created a deep learning GCN algorithm and fed the data to the classifier, and prepared for evaluation. The final part is to test the data to validate the model. As a result, the created model gave an accuracy of 98.2%, and the precision of 98.5% [2].

Egozi explained that phishing emails are still a big problem. To solve the problem, the authors came up with a model that had new features. The two steps of the proposed model are machine learning and feature extraction. The model uses 26

features to figure out if the emails are phishing or legitimate (ham). Different machine learning models work best with different sets of data. The study tried out 17 models, and 14 of them worked well. The authors utilized data from the IWSPA competition for the dataset. The training dataset contains 735 phishing emails and 3865 ham emails. The testing dataset contains 475 phishing emails and 3824 ham emails. The detector successfully identified 95% of legitimate emails and 80% of phishing attempts [3].

Anirudh explored ways to improve detecting phishing mail by combining Support Vector Machine (SVM), XGBoost algorithms, and logistic regression to create a better system. These algorithms do independent analyses of the data and make their own predictions. The combination layer then compiles these forecasts. The combination of these predictions is then used to train a meta learner. It learns how to use these predictions most effectively by figuring out the appropriate weights for them throughout the training process. The dataset has 18650 emails. All of them are tagged either "Safe Email" or "Phishing Email". As a result 11322 emails are tagged "Safe Email" and the remaining 7328 emails are tagged "Phishing Email". SVM, Random Forest, Multinomial-Naïve Bayes, and XGBoost algorithms were used to compare the outcomes of the suggested methodology in order to evaluate its effectiveness. Multinomial Naive Bayes (MNB) had an accuracy of 96.22, a precision score of 96.36, and a recall score of 97.36. and an overall F1-score of 96.96. SVM had an accuracy of 96.74%, a precision score of 98.05%, a recall score of 96.52%, and an overall F1-score of 97.28%. XGBoost had an accuracy of 96.05%, a precision score of 97.75%, a recall score of 95.67%, and an overall F1-score of 96.70%. Random Forest had an accuracy of 97.06%, a precision score of 97.92%, a recall score of 97.33%, and an overall F1-score of 97.62%. The accuracy of the proposed method is 97.92%, surpassing the performance of other models. The proposed method had a precision score of 98.43%, a recall of 98.22%, and a 98.32% overall F1-score [4].

Chinta et al. proposed a mixed model that uses both BERT and LSTM to detect phishing emails. The study uses preprocessing steps like tokenization, stemming, and getting rid of HTML tags, stop words, and punctuation to get the email content ready. Using methods like TF-IDF, features are taken out and turned into numerical vectors that ML models can use. The suggested BERT-LSTM model uses BERT to understand context and LSTM to find patterns in sequences. This solves problems like vanishing gradients in RNNs. When put to the test, the model was much better than baseline models like Naïve Bayes, RNN, and SVM, getting 99.55% accuracy and a 99.24 F1-score. Despite high performance, the model needs a lot of labeled data and a lot of computing power, which could make it hard to use in real life [5].

Bountakas et al. investigated the effectiveness of different NLP and ML combinations by using only email body data for phishing detection. The research evaluates three NLP techniques, BERT, Word2Vec, and TF-IDF, for feature extraction, followed by classification using common ML algorithms such as Random Forest (RF), Logistic Regression (LR), Gradient Boosted Trees (GBT), and Decision Trees (DT). Two datasets were constructed: one balanced and one imbalanced (with a 1:10 phishing-to-benign email ratio), using the Enron corpus for benign emails and the Nazario phishing dataset. The results show that Word2Vec did better than both TF-IDF and BERT on both datasets. Word2Vec with RF and LR got more than 95% accuracy on the balanced dataset, while BERT-based models always did worse. Word2Vec with LR got the highest F1 score of 92.41% on the imbalanced dataset, but BERT-based combinations got much lower scores. For example, LR + BERT only got 63.92% F1 score. [6].

Kumar et al. examined the performance of diverse machine learning classifiers in identifying spam emails, utilizing datasets from Kaggle and additional sources. The main dataset has 5,573 email samples, which are divided into two groups: 70% for training and 30% for testing. Using the Scikit-learn framework with different experimental settings, the authors test eight classifiers: Random Forest, Decision Tree, K-Nearest Neighbor (KNN), Naïve Bayes, Support Vector Machine (SVM), AdaBoost, and Bagging. The results show that Naive Bayes achieves the best individual scores up to 0.98 in precision and recall when parameters are adjusted and improved. On the other hand, AdaBoost and Bagging classifiers always get good scores across all metrics, around 0.94–0.95. Naïve Bayes works best in this study, but it has a flaw: it assumes that features are independent, which could make it less reliable in the real world. By combining several classifiers, ensemble methods help reduce variance and overfitting [7].

Patil et al. used both Natural Language Processing (NLP) and machine learning algorithms to find phishing emails with a high level of accuracy. There are three main steps in their method: preprocessing the text, extracting features, and supervised classification. Tokenization, normalization, and getting rid of irrelevant text elements are all standard NLP tasks that happen during the preprocessing stage. This makes it possible to extract discriminative linguistic features from email content. Feature representations are primarily based on statistical text representation methods, such as TF-IDF, which are then used as inputs for machine learning classifiers. To increase the performance of classification, the study applies ensemble learning strategies, specifically Stacking and Majority Voting, by combining multiple machine learning classifiers rather than relying on a single model. When using a labeled dataset of phishing and real emails, ensemble-based methods get better accuracy and F1-scores than single classifiers. The study is based on a large set of public phishing emails that includes more than 80,000 email records from a variety of publicly available sources. The dataset contains both phishing and legitimate emails and is similar in size, variety, and label distribution to Kaggle phishing email datasets that are widely used. The findings show that the combination of NLP-based feature extraction and ensemble learning enhances the reliability of phishing email detection. The Stacking ensemble method works the best of all the ones that were tested. It has an accuracy of 98.89% and very low false positive and false

TABLE I
COMPARATIVE ANALYSIS OF RELATED WORKS IN PHISHING DETECTION

Study	Year	Model	Dataset	NLP / Features	Metrics	Key Results
[1]	2023	ML + DL	Enron	Tokenization, Padding	Acc, F1	Acc: ∼98%
[2]	2021	GCN & DL	CLAIR (8.5k)	PMI, TF-IDF, Graph	Acc, F1, FPR	Acc: 98.2%
[3]	2018	ML Ensemble	IWSPA	Word count, Punctuation	TPR	TPR: 83%
[4]	2024	Stacking (SVM, XGBoost)	Kaggle (18.6k)	TF-IDF, Stemming	Acc, F1	Acc: 97.92%
[5]	2025	BERT-LSTM	Hybrid (Spam Assassin, UCI)	BERT, Self-attention	Acc, F1	Acc: 99.55%
[6]	2021	ML + NLP	Enron & Nazario	Lemmatization, Chi-Sq	Acc, AUC	Acc: 98.95%
[7]	2020	NB, SVC, KNN	Kaggle Spam	BOW, Stemming	Acc	Acc: 98%
[8]	2025	Ensemble	Kaggle (82k)	TF-IDF, CountVect	Acc, F1	Acc: 98.89%
Our Study	**2026**	**Agentic MCP**	**Dual Dataset**	**Automated NLP**	**Acc, AUC**	**Acc: 98.76%**

negative rates of 0.01. But these performance improvements come at the cost of more complicated calculations, since ensemble models need to be trained for much longer periods of time, more than 2000 seconds [8].

III. PROPOSED METHODOLOGY & IMPLEMENTATION

This study proposes an agent-guided system for building and evaluating phishing email detection pipelines with reduced manual intervention. The agent is designed to operate within predefined constraints, including a predefined set of candidate models, fixed evaluation metrics, and structured output formats. This design is meant to make it easier to reproduce results, reduce manual effort, and let datasets with different schemas be compared in a consistent way.

A. System Architecture

The system uses a client-server design based on the Model Context Protocol (MCP). This protocol acts as a bridge, allowing the AI agent to interact with the computational environment and execute code securely. The overall workflow is illustrated in Fig. 1.

1) The Agent (Client): In this study, the Google Gemini 2.5 Flash model is the main part of the agent's intelligence. It breaks down big objectives into smaller, more specific tasks and keeps track of the whole workflow. The agent keeps track of information about past steps and results, which helps it make smart choices while the task is being carried out. This helps the system get better results by using information from previous steps.

2) The Runtime (Server): Since AI models cannot directly execute code in a local environment, a Node.js-based server is used to handle execution tasks. This server provides the agent with a set of predefined tools: auto_ml_agent: This allows the agent to write and run Python scripts for training and evaluating multiple machine learning models.

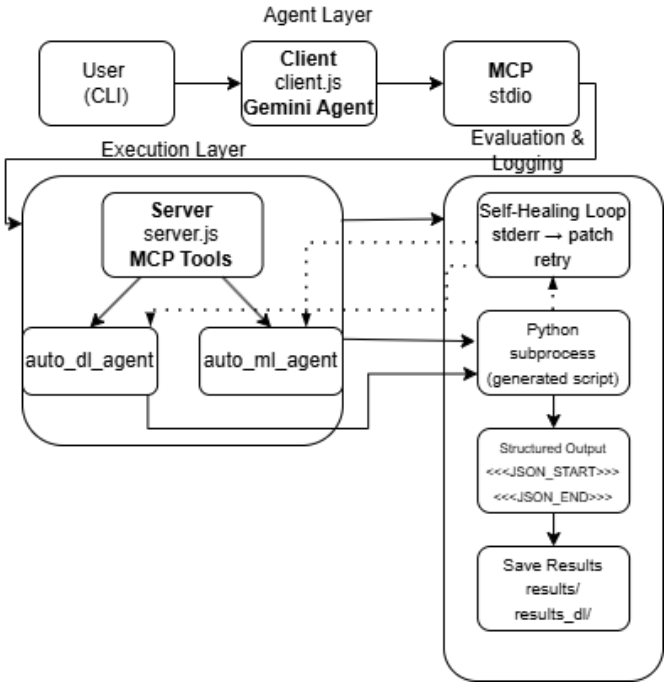


Fig. 1. Agent-guided MCP architecture for automated phishing detection.

auto_dl_agent: This is for designing deep learning models. It lets the agent write complex code using TensorFlow and Keras.

3) The Healing Loop: A key component of this architecture is the Self-Healing Loop. Code generated by large language models may contain runtime errors, such as tensor shape mismatches or deprecated library calls. To address this, the server captures the standard error (stderr) output from failed Python executions. The captured error trace is then returned

to the agent, which generates a corrective patch describing the necessary changes to fix the error. This patch is applied to the code, and the execution is re-tested. If the problem continues then the whole script is rewritten. If the error remains unresolved, the execution is terminated. This process improves the system's robustness and reduces failures during pipeline generation.

B. Operational Data Flow and Interaction Protocol

The Model Context Protocol (MCP) acts as an intermediary layer that standardizes communication between the agent and the execution environment, ensuring secure and structured code execution.

To increase the transparency and reproducibility of the study, the interaction between the agent and the runtime follows a clear, step-by-step process:

1) **User Input:** User provides dataset path and selects mode (ML/DL).
2) **Planning:** Agent identifies the data schema (text vs. label columns).
3) **Code Generation:** The agent generates a Python script that strictly follows the predefined experimental protocol.
4) **Execution:** The generated script is dispatched to the MCP server, where it is executed in an isolated environment.
5) **Feedback Loop:** If execution fails, the MCP runtime returns error logs (stderr) to the agent. The agent analyzes these logs and initially generates a corrective patch. If the error persists, it performs a full script rewrite. This strict recovery protocol prevents infinite loops.
6) **Final Evaluation:** Successful execution produces a standardized JSON record.

C. Standardized Output for Reproducibility

A critical requirement for comparing datasets with different schemas is a consistent output format. Regardless of the internal dataset structure, the system returns a fixed JSON schema:

```
{
  "best_model": "RandomForest",
  "test_accuracy": 0.9863,
  "f1_score": 0.9868
}
```

This standardization ensures that the results reported in Section V are directly comparable across both Dataset 1 and Dataset 2.

D. Agent-Guided Pipeline Generation

The proposed system is an agent-guided workflow that reduces manual work while keeping the experiment design under explicit constraints. The user still chooses the experiment mode (AutoML or AutoDL) and provides the dataset path. After that, the agent generates a complete Python training script that follows fixed rules (preprocessing, models to test, metrics to compute, and a strict JSON output format). The goal is to automate repetitive engineering steps (data loading, column handling, model training, evaluation, and reporting) while maintaining a consistent research protocol.

The workflow consists of three main steps:

1) Step 1: Data Analysis: Before training any model, the agent analyzes the dataset file (CSV). It scans the column names to determine which column contains the email text and which one contains the label (Phishing vs. Safe). It looks for common names like "content," "body," or "message" for text, and "spam" or "label" for the class. This allows the system to work with many different datasets without needing a human to rename columns.

2) Step 2: Agent-Guided Machine Learning (AutoML): In AutoML mode, the agent generates a machine learning text-classification pipeline. The main idea is: *convert text into numeric features, then train multiple standard classifiers, and compare them using the same evaluation protocol.*

First, the script uses TF-IDF vectorization to convert each email into a numerical vector. TF-IDF gives more weight to words that are common in one email but less common in the whole dataset. This helps highlight terms commonly used in phishing messages.

Next, the script trains and evaluates several widely used baseline classifiers for text classification: Logistic Regression, Multinomial Naive Bayes, Random Forest, and Linear SVM. To reduce overfitting and make model comparisons more reliable, the system uses 5-fold cross-validation during model evaluation. After selecting the best-performing model according to the evaluation protocol, the script reports metrics on a held-out test set and returns results in a standardized JSON structure.

3) Step 3: Agent-Guided Deep Learning (AutoDL): For complex pattern recognition, the agent generates Deep Learning architectures. It implements a Keras-based preprocessing layer that tokenizes the email body and pads sequences to a fixed length ($L = 200$). The agent then designs and trains multiple architectures in parallel: Recurrent Neural Networks (RNNs): LSTM, BiLSTM, and GRU for sequence modeling. Convolutional Neural Networks (CNNs): 1D-CNN for local n-gram feature extraction, and a predefined Hybrid (CNN+Dense) architecture that combines convolutional feature extraction with fully connected Dense layers for binary classification.

After evaluating these predefined baselines, the agent autonomously designs an additional custom architecture, referred to as `auto_model`. The agent constructs this model autonomously under explicit constraints: the architecture must structurally differ from all mandatory baselines, may incorporate Attention mechanisms, CNN–RNN hybrids, or multi-branch convolutional networks, and must be evaluated under identical experimental conditions. All six architectures are assessed using the same performance metrics to ensure a fair and reproducible comparison.

E. Feature Extraction & Explainability

Machine learning models needed a reliable way to "read" and understand unstructured emails. To achieve this, the entire ML pipeline is built around TF-IDF (Term Frequency-Inverse Document Frequency) vectorization. TF-IDF serves as our primary NLP engine: it converts raw sentences into meaningful numbers by identifying which words are uniquely important to a suspicious email, while naturally ignoring common everyday English words. By extracting the highest-scoring TF–IDF features from linear models (e.g., LinearSVC and Logistic Regression), the most influential phishing-related keywords can be identified.

When analyzing the actual empirical results from the agent's execution, the semantic audit proved highly successful across both datasets. In Dataset 1, the model correctly assigned the highest predictive weights to classic spam and financial fraud indicators such as *"investment"*, *"money"*, *"remove"*, and *"reply"*. Similarly, in the much larger Dataset 2, the extraction highlighted credential-harvesting terminology and malicious routing indicators with terms like *"account"*, *"click"*, *"company"*, and *"http"*. Identifying these specific lexical markers demonstrates that the ML agent did not simply memorize formatting artifacts; rather, it effectively comprehended the vocabulary of a cyberattack, yielding both high predictive accuracy and total explainability.

F. Configuration and Computational Environment

To ensure the reproducibility of automated pipelines, the hardware and software context used to generate and train models is formally defined. Table II details the specific configurations of the computational environment, including the Node.js agent stack and the Python ML dependencies.

TABLE II
COMPREHENSIVE HARDWARE AND SOFTWARE ENVIRONMENT

Component Category	Specification / Version
Hardware Specifications	
Processor (CPU)	Intel(R) Core(TM) i7-7700HQ @ 2.80GHz
Memory (RAM)	16 GB DDR4
Graphics Card (GPU)	NVIDIA GeForce GTX 1060 (Max-Q)
Machine Learning Stack (Python)	
Python Runtime	3.13
TensorFlow / Keras	2.20.0 / 3.11.2
Scikit-learn	1.7.2
Pandas	2.2.3
NumPy	2.2.3
Agentic Framework Stack (Node.js)	
Node.js Runtime	v22.14.0
MCP SDK	1.20.2
Google Generative AI	0.24.1 (Gemini Integration)
Zod	3.22.4 (Schema Validation)

IV. DATASET DETAILS

For this study, 2 different datasets are used. Both datasets are obtained from Kaggle and provide labeled email samples suitable for binary classification tasks.

A. Dataset 1

The first dataset used in this research is the Phishing Email Detection, created by Subhadeep Chakraborty and published in 2023 on Kaggle [9].

This dataset contains 18,650 email samples labeled as either phishing or legitimate. Each record consists of two main attributes:

- **Email Text** – Content of the email
- **Email Type** – Indicating if the email is phishing or not

The dataset is designed for phishing email detection research and supports binary text classification, where the goal is to identify malicious phishing emails among legitimate messages.

According to the dataset statistics, approximately 61% of the emails are labeled as safe emails, while 39% are labeled as phishing emails.

B. Dataset 2

The second dataset used in this study is the Phishing Email Dataset, published in 2024 on Kaggle by Naser Abdullah Alam [10] [11].

This dataset was constructed by combining several publicly available phishing email datasets to create a larger corpus suitable for machine learning research. The combined dataset integrates information from multiple well-known email datasets, including the Enron, Nazario, CEAS, Ling, Nigerian Fraud, and SpamAssassin datasets.

The original datasets contain several types of information, such as email subject lines, email body content, sender information, recipient information, timestamps, and classification labels. However, in the final published version of the dataset, these fields were merged into a single textual representation.

The resulting dataset contains approximately 82,000 email samples, consisting of 39,595 legitimate and 42,891 spam emails.

For the purposes of this study, the dataset contains two primary attributes:

- **text_combined**: A single textual field that combines the contents of the original email components, such as subject, body, and other metadata, into one text representation.
- **label**: A binary classification label tells whether the email is phishing or safe.

By combining multiple parts of the original email structure into the text_combined column, the dataset provides a unified textual input that can be directly used for natural language processing and machine learning models.

The dataset contains over 82,000 total records. The larger scale allows the proposed automated pipeline system to be evaluated on a more diverse and complex phishing email dataset.

V. RESULTS

A. Experimental Setup

All experiments use a stratified 80/20 train–test split. For machine learning (ML), emails are represented using TF–

IDF features (max_features = 5000). For deep learning (DL), texts are tokenized and padded to a fixed sequence length of $L = 200$. Performance is measured by accuracy, precision, recall, and F1-score, where the phishing class is treated as the positive class. Confusion matrices are reported in the form $\begin{bmatrix} \mathrm{TN} & \mathrm{FP} \\ \mathrm{FN} & \mathrm{TP} \end{bmatrix}$.

B. Evaluation Metrics

To evaluate phishing email classification performance, we report standard metrics derived from the confusion matrix. In this study, the phishing class is treated as the positive class.

a) Confusion Matrix Terms: True Positives (TP) represent the successful categorization of phishing emails, and True Negatives (TN) indicate legitimate messages that the classifier correctly recognizes as safe. On the other hand, False Negatives (FN) are real phishing attacks that are mistakenly categorized as benign, and False Positives (FP) happen when safe messages are mistakenly categorized as malicious

b) Accuracy: Accuracy measures how often the classifier correctly predicts both positive and negative cases

$$\mathrm{Acc} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}}. \tag{1}$$

c) Precision: Precision measures how reliable phishing alerts are:

$$\mathrm{Prec} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}. \tag{2}$$

d) Recall: Recall (sensitivity) measures how many actual phishing emails are detected:

$$\mathrm{Rec} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}. \tag{3}$$

In cybersecurity, recall is important because false negatives correspond to phishing emails that bypass detection.

e) F1-Score: The harmonic mean of recall and precision gives the F1-score:

$$\mathrm{F1} = \frac{2 \cdot \mathrm{Prec} \cdot \mathrm{Rec}}{\mathrm{Prec} + \mathrm{Rec}}. \tag{4}$$

F1 is useful when both false positives and false negatives are important, and a single balanced metric is desired.

f) Training Time: In addition to predictive performance, we report training time (seconds) to compare computational cost. This is particularly relevant for deep learning models and for automated pipeline execution.

C. Model Performance

The following section presents the results for both ML and DL pipelines, evaluated independently on Dataset 1 (18.6k samples) and Dataset 2 (82k samples).

As shown in Table III, LinearSVC achieves the best overall performance on Dataset 1, obtaining the highest test accuracy (96.84%) along with strong precision and recall values, while maintaining a low training time.

For Dataset 2, RandomForest achieves the highest test accuracy (98.63%) and F1-score. However, LinearSVC provides a competitive performance with significantly lower training time, making it a more efficient alternative.

Overall, increasing the dataset size leads to consistent performance improvements across all models.

Table IV presents the performance of deep learning models. The Hybrid (CNN+Dense) architecture achieves the best performance on both datasets, reaching 97.00% accuracy on Dataset 1 and 98.76% on Dataset 2.

In addition to accuracy, the Hybrid model maintains a strong balance between precision and recall, while also offering relatively low training time compared to recurrent architectures such as LSTM and GRU.

These results indicate that combining convolutional feature extraction with dense layers provides an effective and efficient architecture for phishing detection.

The confusion matrices for the best-performing models are shown in Table V.

Across both datasets, the number of misclassifications remains low, with false positives and false negatives being limited. This indicates that the models are able to generalize well and maintain balanced performance between phishing and legitimate classes.

D. Discussion on Model Performance

The experimental results reveal that the performance of both models on both datasets highlighting consistent performance between ML and deep learning DL approaches.

In the ML section, the models give competitive performance because they use TF-IDF. TF-IDF captures the highly distinctive and repetitive keywords (e.g., “account”, “click”, “verify”). Linear models such as LinearSVC can exploit these high-dimensional sparse features efficiently, leading to strong classification performance with very low computational cost. This explains why LinearSVC achieves competitive accuracy (96.84% on Dataset 1 and 98.51% on Dataset 2) while maintaining significantly faster training times compared to all other models.

On the other hand, DL models learn contextual and sequential representations from raw text. Architectures such as LSTM, GRU, and BiLSTM are designed to capture long-range dependencies and semantic relationships within email content. However, on smaller datasets (Dataset 1), these models may not fully utilize their representational capacity, resulting in only marginal improvements over ML approaches. As the dataset size increases (Dataset 2), DL models benefit from the larger training corpus, allowing them to better generalize and slightly outperform traditional ML models.

1) Why the Hybrid Model Performs Best in Accuracy-Based Evaluation: The Hybrid (CNN+Dense) architecture consistently achieves the highest performance across both datasets. This can be attributed to its ability to combine local feature extraction with global feature integration.

The convolutional layers (CNN) act as n-gram feature extractors, identifying important local patterns such as short phrases commonly found in phishing emails (e.g., “click here”, “verify account”). These extracted features are then passed to

TABLE III
CLASSICAL ML PERFORMANCE METRICS — DATASET 1 (18.6K) AND DATASET 2 (82K).

Dataset	Model	CV Acc	CV Std	Test Acc	Precision	Recall	F1-Score	Time (s)
D1 (18.6k)	LogisticRegression (TF-IDF+LR)	0.9657	0.0046	0.9601	0.9440	0.9550	0.9495	2.71
	MultinomialNB (TF-IDF+MNB)	0.9592	0.0045	0.9499	0.9312	0.9420	0.9366	**0.22**
	RandomForest (TF-IDF+RF)	0.9602	0.0049	0.9558	0.9386	0.9495	0.9440	150.07
	LinearSVC (TF-IDF+SVM)	**0.9714**	0.0041	**0.9684**	**0.9475**	**0.9734**	**0.9603**	1.68
	GradientBoosting (TF-IDF+GBT)	0.9342	**0.0039**	0.9273	0.8965	0.9216	0.9088	1012.21
D2 (82k)	LogisticRegression (TF-IDF+LR)	0.9817	0.0006	0.9825	0.9812	**0.9853**	0.9832	11.53
	MultinomialNB (TF-IDF+MNB)	0.9615	0.0011	0.9617	0.9786	0.9471	0.9626	**3.46**
	RandomForest (TF-IDF+RF)	0.9842	0.0011	**0.9863**	**0.9884**	0.9852	**0.9868**	178.40
	LinearSVC (TF-IDF+SVM)	**0.9849**	**0.0006**	0.9851	0.9865	0.9850	0.9857	3.70
	GradientBoosting (TF-IDF+GBT)	0.9447	0.0006	0.9435	0.9242	0.9710	0.9470	632.33

TABLE IV
DEEP LEARNING PERFORMANCE METRICS — DATASET 1 (18.6K) AND DATASET 2 (82K).

Dataset	Model	Test Acc	Precision	Recall	F1-Score	Time (s)
D1 (18.6k)	LSTM (Embedding+LSTM)	0.9491	0.9293	0.9420	0.9356	887.70
	BiLSTM (Embedding+BiLSTM)	0.9603	0.9654	0.9325	0.9486	460.32
	GRU (Embedding+GRU)	0.9630	**0.9770**	0.9277	0.9517	561.73
	CNN (Embedding+Conv1D+GMP)	0.9697	0.9477	0.9768	0.9620	**247.77**
	Hybrid (CNN+Dense)	**0.9700**	0.9489	0.9761	**0.9623**	248.96
	auto_model (CNN+LSTM)	0.9668	0.9312	**0.9884**	0.9590	369.22
D2 (82k)	LSTM (Embedding+LSTM)	0.9848	0.9831	0.9878	0.9854	1645.05
	BiLSTM (Embedding+BiLSTM)	0.9858	0.9844	0.9883	0.9864	674.18
	GRU (Embedding+GRU)	0.9866	**0.9906**	0.9836	0.9871	830.57
	CNN (Embedding+Conv1D+GMP)	0.9867	0.9892	0.9851	0.9872	209.32
	Hybrid (CNN+Dense)	**0.9876**	0.9874	**0.9887**	**0.9881**	**170.45**
	auto_model (CNN+CNN+GRU)	0.9850	0.9905	0.9807	0.9855	536.41

TABLE V
CONFUSION MATRICES FOR BEST ML AND DL MODELS ACROSS BOTH DATASETS.

(a) D1 — LinearSVC (TF-IDF+SVM)

		Predicted: Phishing	Predicted: Legitimate
Actual	**Phishing**	1427	39
	Legitimate	79	2185

(b) D2 — RandomForest (TF-IDF+RF)

		Predicted: Phishing	Predicted: Legitimate
Actual	**Phishing**	8452	127
	Legitimate	99	7820

(c) D1 — Hybrid (CNN+Dense)

		Predicted: Phishing	Predicted: Legitimate
Actual	**Phishing**	1431	35
	Legitimate	77	2187

(d) D2 — Hybrid (CNN+Dense)

		Predicted: Phishing	Predicted: Legitimate
Actual	**Phishing**	8482	97
	Legitimate	108	7811

fully connected Dense layers, which combine these patterns and make the final classification decision.

This combination enables the model to capture both:

- Key phrases (via CNN)
- Combined contextual understanding (via Dense layers)

As a result, the Hybrid model achieves a balanced improvement in both precision and recall, leading to the highest F1-scores and overall accuracy (up to 98.76%). Compared to pure RNN-based models, which focus primarily on sequence modeling, the Hybrid architecture provides a more reliable representation for phishing detection tasks.

Another key observation is the trade-off between performance and computational efficiency. While DL models achieve the highest accuracy, they require significantly longer training times (hundreds to thousands of seconds). In contrast, LinearSVC provides near-optimal performance with minimal training time (as low as 1.68 seconds), making it highly suitable for real-time or resource-constrained environments.

Overall, the results suggest that while DL models offer marginal performance gains, traditional ML models remain

highly competitive, particularly when efficiency is a critical requirement.

E. ROC Analysis

The classification performance of the proposed agent-guided framework was further evaluated using Receiver Operating Characteristic (ROC) curves. Figures 2 through 5 illustrate the performance of ML and DL models across both datasets.

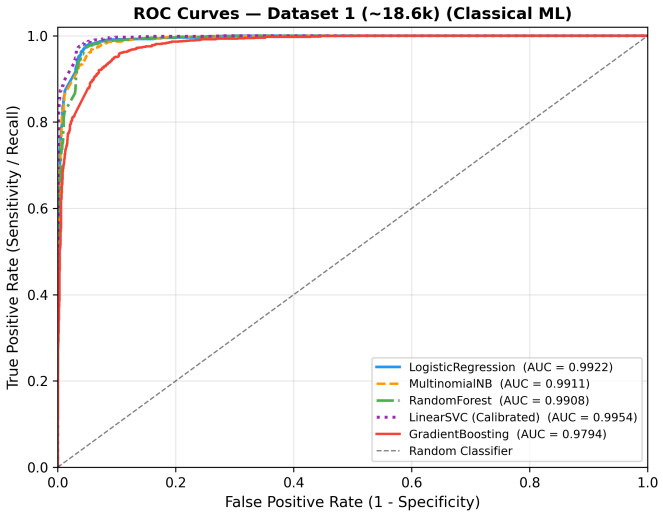


Fig. 2. ROC curves for ML models on Dataset 1 (18.6k samples). LinearSVC achieves the highest AUC of 0.9954.

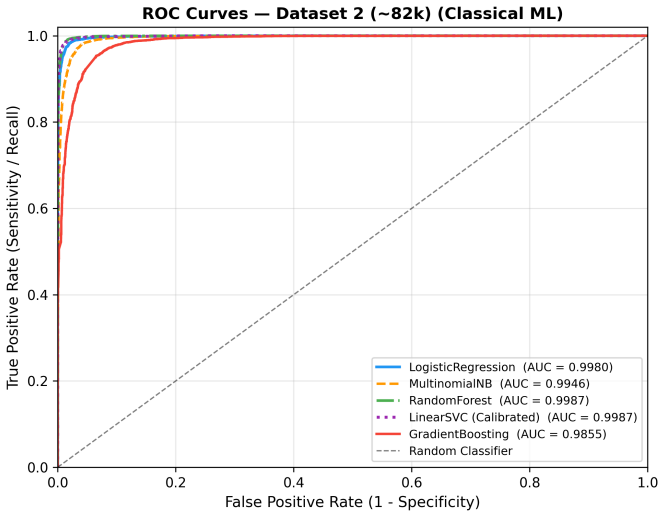


Fig. 3. ROC curves for ML models on Dataset 2 (82k samples). Both RandomForest and LinearSVC demonstrate near-perfect discriminative ability with an AUC of 0.9987.

Across all experiments, the ROC curves are positioned close to the top-left corner, indicating strong discriminative capability for both ML and DL models. The corresponding AUC values further confirm that all models achieve near-perfect classification performance.

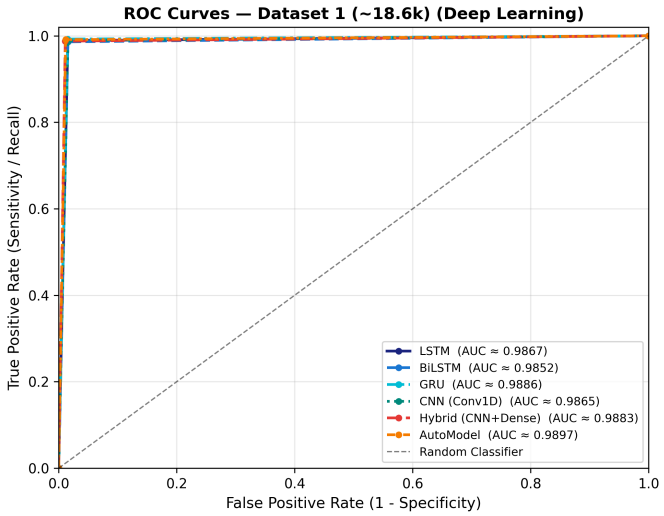


Fig. 4. ROC curves for Deep Learning models on Dataset 1. The AutoModel (CNN+LSTM) achieves the highest AUC of 0.9897.

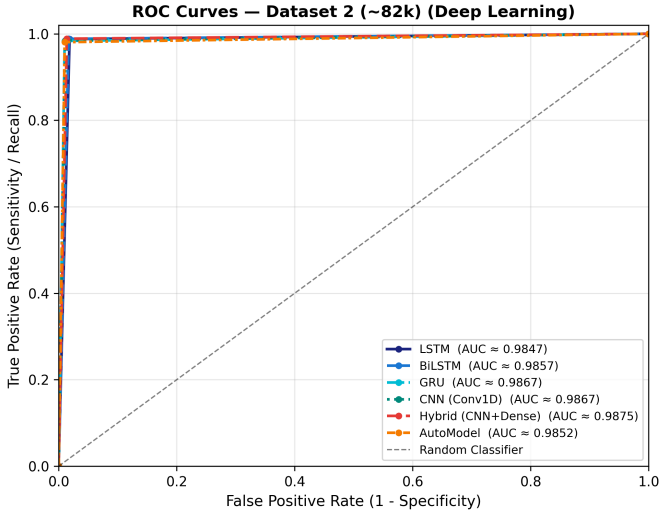


Fig. 5. ROC curves for Deep Learning models on Dataset 2. The Hybrid model achieves a balanced AUC of 0.9875.

1) ML Model ROC Performance: For Dataset 1, LinearSVC achieves the highest AUC (0.9954) among ML models, demonstrating that TF–IDF features combined with linear decision boundaries are highly effective in separating phishing and legitimate emails. This suggests that the dataset contains strong lexical patterns that can be captured without requiring complex nonlinear modeling.

Similarly, in Dataset 2, both RandomForest and LinearSVC achieve near-perfect AUC scores (0.9987), indicating that the classification problem becomes even more separable with increased data. The improvement in AUC values reflects better generalization and reduced variance due to the larger dataset size.

2) DL Model ROC Performance: For deep learning models, the ROC curves also demonstrate strong performance,

although with slightly lower AUC values compared to ML models. On Dataset 1, the AutoModel (CNN+LSTM) achieves the highest AUC (0.9897), suggesting that combining convolutional feature extraction with sequence modeling improves discriminative capability. However, the difference between DL models remains marginal, indicating that most architectures learn similar decision boundaries on smaller datasets.

On Dataset 2, the Hybrid (CNN+Dense) model achieves the most balanced performance (AUC = 0.9875), maintaining high sensitivity and specificity across different classification thresholds. This confirms that the hybrid architecture provides stable and consistent predictions rather than optimizing for a single operating point.

3) Why ML Achieves Higher AUC Than DL in ROC Analysis: Despite the overall strong performance of both approaches, traditional ML models consistently achieve higher AUC values than DL models, and this difference is most evident on the larger Dataset 2.

This outcome is closely related to the nature of phishing emails. Phishing messages typically rely on a consistent set of suspicious words and phrases, such as urgent warnings and requests for personal information. TF-IDF captures these patterns directly, allowing linear classifiers like LinearSVC to separate the two classes confidently across all decision thresholds, which is reflected in higher AUC scores.

DL models, by contrast, are designed to learn deeper contextual relationships between words. While this capability is useful in many NLP tasks, it does not offer a clear advantage here, since the distinguishing features of phishing emails are largely word-level rather than context-dependent. In addition, on the smaller Dataset 1, DL models have less data to learn from, which can lead to less reliable probability estimates and, consequently, lower AUC values even when accuracy remains competitive.

Despite this, DL models remain strong performers overall and may offer better generalization in more diverse or complex email datasets.

VI. Our Contributions

The primary objective of this study is to move beyond static model evaluation and introduce an autonomous framework for phishing detection. Unlike existing literature that focuses on manual feature engineering and fixed model architectures, our work makes the following original contributions:

- **High-Accuracy Autonomous Detection:** The proposed agent-guided framework achieves a peak test accuracy of **98.76%** using automated hybrid architectures, matching or exceeding the performance of manually fine-tuned models reported in recent literature.
- **Optimized Performance-Efficiency Trade-off:** While recent ensemble-based studies report high accuracy with training times exceeding 2000 seconds, the automated LinearSVC pipeline introduced in this work delivers a highly competitive **98.51% accuracy in only 3.70 seconds**, representing a significant advancement in computational efficiency for real-time phishing detection.
- **Self-Healing Deployment:** Self-healing mechanism is introduced that systematically identifies and repairs runtime errors and tensor mismatches in generated code. This ensures the successful deployment of high-accuracy pipelines without human intervention.
- **Standardized Agentic Orchestration:** This study provides a novel Model Context Protocol (MCP) based architecture that standardizes the experimental protocol across diverse datasets, effectively minimizing human bias and manual engineering effort traditionally required in phishing detection research.

VII. Conclusion

This study successfully demonstrated the effectiveness of an agent-guided Model Context Protocol (MCP) framework for automated phishing email detection. By integrating both Machine Learning and Deep Learning pipelines, the system significantly reduces the manual engineering effort typically required for data preprocessing, model selection, and cross-validation, while ensuring scientific reproducibility.

The framework was evaluated on two distinct datasets using both Machine Learning (ML) and Deep Learning (DL) approaches. The experimental results demonstrated that while ML models, particularly LinearSVC, offer an exceptional balance of high accuracy (98.51%) and rapid training times (3.70s) on large corpora, DL architectures ultimately achieved the highest predictive performance. Specifically, the Hybrid (CNN+Dense) model attained a peak test accuracy of 98.76% and an F1-score of 98.81% on the combined 82k dataset. These findings validate that agent-orchestrated NLP pipelines can provide highly reliable, scalable, and autonomous defenses against evolving phishing threats.

The MCP server currently has only two tools, one for machine learning and one for deep learning. Future work could introduce additional, more specialized tools, where each tool is responsible for a specific stage of the pipeline, such as data cleaning, feature extraction, model training, or result reporting. This would give the agent finer control over each step of the workflow.

References

[1] N. S. Paradkar, "Phishing email's detection using machine learning and deep learning," in *2023 3rd International Conference on Advances in Computing, Communication, Embedded and Secure Systems (ACCESS)*. IEEE, 2023, pp. 160–162.

[2] A. Alhogail and A. Alsabih, "Applying machine learning and natural language processing to detect phishing email," *Computers & Security*, vol. 110, p. 102414, 2021.

[3] G. Egozi and R. Verma, "Phishing email detection using robust nlp techniques," in *2018 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 2018, pp. 7–12.

[4] S. Anirudh, P. R. Nishant, S. Baitha, and K. D. Kumar, "An ensemble classification model for phishing mail detection," *Procedia Computer Science*, vol. 233, pp. 970–978, 2024.

[5] P. C. R. Chinta, C. S. Moore, L. M. Karaka, M. Sakuru, V. Bodepudi, and S. R. Maka, "Building an intelligent phishing email detection system using machine learning and feature engineering," *European Journal of Applied Science, Engineering and Technology*, vol. 3, no. 2, pp. 41–54, 2025.

[6] P. Bountakas, K. Koutroumpouchos, and C. Xenakis, "A comparison of natural language processing and machine learning methods for phishing email detection," in *Proceedings of the 16th International Conference on Availability, Reliability and Security*, 2021, pp. 1–12.

[7] N. Kumar, S. Sonowal *et al.*, "Email spam detection using machine learning algorithms," in *2020 Second International Conference on Inventive Research in Computing Applications (ICIRCA)*. IEEE, 2020, pp. 108–113.

[8] D. Patil, "Integrating nlp and ensemble methods for large-scale phishing email detection," *Intelligent Decision Technologies*, vol. 19, no. 4, pp. 2123–2144, 2025.

[9] S. Chakraborty, "Phishing email detection," 2023. [Online]. Available: https://www.kaggle.com/dsv/6090437

[10] N. A. Alam, "Phishing email dataset," 2024. [Online]. Available: https://www.kaggle.com/ds/5074342

[11] A. Al-Subaiey, M. Al-Thani, N. A. Alam, K. F. Antora, A. Khandakar, and S. A. U. Zaman, "Novel interpretable and robust web-based ai platform for phishing email detection," *Computers and Electrical Engineering*, vol. 120, p. 109625, 2024.